

Informe TQS: F1 2D

S'expliquen el conjunt de proves realitzades en el projecte d'un joc de carreres amb temàtica de Fórmula 1. El programa té dos modes de joc, "Qualy" (per practicar) i "Race" per fer una carrera contra altres cotxes IA. Té sectors, temps per sectors, marcador de posicions en temps real, so amb efectes reals (mític "box-box"), slipstream i comprovador de col·lisions de mapa, entre altres.

Les proves realitzades es detallen per categories i fitxers en la següent llista:

(Existeixen proves fetes per la part visual, de so i d'assets com LapUI i OffTrackOverlay, però com no es demanen no es tindran en compte en aquest informe. Es poden mirar en el projecte directament dins dels següents fitxers:

Dins del projecte es troben a CarDisplayTest.java que s'encarrega del menú principal i visualització de mapa. LapUITest.java que s'encarrega de les imatges de sectors, temps per volta i font de text personalitzada.

OfftrackOverlay.java que tracta sobre els sons i les imatges que surten quan el cotxe se surt de pista.

)

Proves TDD:

CONTROLADORS:

AIControllerTest.java

(Hi ha poc TDD, ja que la resta es necessita principalment la utilització de mock objects, per tant, gran part del TDD, com a definició formal, es troba a l'apartat de proves amb mock objects)

nProva	Nom i lloc	Què comprova?
1	testConstructor()@Línia69	Inicialització del controlador
2	testSetAIDifficulty()@Línia78	Verifica que el mètode ajusti correctament la dificultat de les IA
3	testReset()@Línia89	Comprova que el mètode reset, restableixi l'estat del controlador
4	testGetAIPosition()@Línia99	Verificar que els cotxes IA tenen posicions
5	testAngleCalculationInRange()@Línia108	Validar que el càlcul del waypoint sempre estigui entre 0 i 360 graus

CarControllerTest.java

(La gran majoria de tests dissenyats per a aquesta classe representen més a particions i valors que un TDD. Aquests són els primers tests que es van realitzar abans de fer caixa negra)

nProva	Nom i lloc	Què comprova?
6	testProcessInput()@Línia55	Comprova que un sol input es processi de forma correcta
7	testProcessMultipleInputsWD()@Línia68	Verifica que múltiples inputs a la vegada es processin adequadament fent que el jugador pugui realitzar dos moviments a la vegada
3	testProcessMultipleInputsWA()@Línia78	Mateixa verificació que el test anterior però amb un altre set d'inputs
4	testProcessMultipleInputsSD()@Línia86	Mateixa verificació que el test anterior però amb un altre set d'inputs
5	testProcessMultipleInputsSA()@Línia94	Mateixa verificació que el test anterior però amb un altre set d'inputs
6	testNoKey()@Línia102	Verifica que no passi res si no hi ha cap input

MapControllerTest.java

(A tenir en compte:

ESTATS DE JOC = IDLE, RUNNING, OFF_TRACK, RESULT, INVALID_LAP, COUNTDOWN, RACE_FINISHED; Invalid = córrer contra direcció, Off-track = sortir de pista

COLORS DE SECTORS = NONE, GREEN, ORANGE, PURPLE
)

nProva	Nom i lloc	Què comprova?
7	testGetters()@Línia75	Comprovar que en inicialitzar, el mapa s'agafa correctament i és el mateix
8	testStartLap@Línia84	En travessar per línia de meta, l'estat ha de passar a "RUNNING".
9	testCheckpoint()@Línia92	En començar una volta i passar per un checkpoint, marcar com a "passat"
10	testLapCompletion()@Línia101	En travessar per línia de meta amb tots els checkpoints, es completa la volta

11	testLapNotCompleted()@Línia113	En travessar per línia de meta sense tots els checkpoints, la volta no es completa
12	testOffTrackStopsLap()@Línia122	Sortir de pista implica parar el temps de volta
13	testReset()@Línia134	Comprovar que reinicia tot l'estat del mapa per tornar a donar una volta o per començar a jugar
14	testResultToRunningTransition()@Línia146	En acabar una volta, s'han d'ensenyar els temps i sectors i tornar a començar una nova volta
15	testFinishLineAfterInvalidLap()@Línia163	En passar per línia de meta independentment de l'estat ha de tornar a començar la volta
16	testEnteringSameCheckpoint()@Línia177	En tornar a passar per un checkpoint ja comptat, no l'ha de tornar a mirar
17	testLastSectorRecorded()@Línia188	En passar per línia de meta amb l'últim sector ja establert no hauria de comptar-se dues vegades
18	testRunningWithSecondSectorAlreadyRecorded() @Línia206	Mateix que test anterior però amb el segon sector
19	testRecordSectorInvalidIndex()@Línia224	Un sector que no existeix no hauria de tenir temps d'establir
20	testRecordSectorPurpleSet()@Línia236	Establir un sector en lila
21	testPassedAllCheckpointsTrue()@Línia247	Verificar el funcionament del flag de passar per tots els checkpoints amb el cotxe (afegit més tard per coverage)
22	testPassedAllCheckpointsFalseSize()@Línia260	Verificar el funcionament del flag de NO passar per tots els checkpoints i saltar-se checkpoints amb el cotxe (afegit més tard per coverage)
23	testPassedAllCheckpointsFalse()@Línia269	Verificar el funcionament del flag de NO passar per tots els checkpoints amb el cotxe (afegit més tard per coverage)
24	testSumIndexZero()@Línia280	Verificació de la funció que suma els temps de sectors
25	testSumWithinBounds@Línia287	Verificació de la funció que suma els temps de sectors
26	testSumIndexGreaterThanLength()@Línia294	Verificació de la funció que suma els temps de sectors
27	testPurpleConditionNotTriggered@Línia301	Verificació funcionament colors de sectors (afegit més tard per coverage)

28	testInvalidateLap()@Línia315	Verificar el coverage de la funció invalidate lap i els seus components (afegit més tard)
29	testStartQualyMode()@Línia339	El joc s'ha d'inicialitzar correctament quan es clica el botó de "Qualy"
30	testOffTrackRacePenalty@Línia351	En sortir de pista s'ha d'aplicar una penalització
31	testOffTrackRaceNoDoublePenalty()@Línia366	La penalització només s'aplica una vegada per volta
32	testOffTrackRaceBackRunning()@Línia380	En carrera, en sortir de pista i tornar a la pista, l'estat ha d'estar en "RUNNING", en Qualy no per tal que no compti el fet de fer la volta
33	testOffTrackRaceTimerBackRunning@Línia392	Igual que el test anterior, però per coverage del timer del asset que surt en sortir de pista (afegit coverage més tard)
34	testEndLapPenalty()@Línia413	Comprovar que si té penalització, no es compti el temps de volta
35	testInvalidHandlerTimer()@Línia443	Igual que el Test num33, però amb l'estat INVALID en comptes de OFFTRACK

MODELS:

AICarTest.java

nProva	Nom i lloc	Què comprova?
36	testAICarCreation()@Línia111	S'ha d'inicialitzar un cotxe amb paràmetres d'equip i de destresa
37	testDistanceCalculation()@Línia137	Calcular correctament la distància a un altre cotxe
38	testSlipstreamDetection()@Línia149	A partir de la segona volta en carrera, el "rebufo" s'activa si es troba a una certa distància del cotxe de davant just darrere seu
39	testAIControllerInitialization()@Línia159	S'inicialitzen 19 cotxes amb habilitats diferents
40	testUpdateAIRacingLineEmpty()@Línia177	Els cotxes necessiten d'una "racing line" per funcionar
41	testApplySlipstreamBehind()@Línia195	Diferents estats de Slipstream (Coverage)
42	testRacingLineResetTargetIndexExceeds()@Línia215	Quan una IA acaba volta ha de tornar a agafar la Racing Line pel primer o segon punt

43	testTargetSwitchCloseToWaypoint()@Línia231	Quan un cotxe IA es troba a prop d'un waypoint pensa a anar al següent
44	testTurningHardLowerVelocity()@Línia276	Si la corba és tancada, la velocitat ha de disminuir en arribar a la corba
45	testStartNewLap()@Línia288	Comprovar que la volta d'un cotxe IA s'inicialitza correctament
46	testRecordSector()@Línia301	Els cotxes IA han de marcar temps en sectors per fer el sector lila del jugador funcionar
47	testCompleteLap()@Línia321	Els cotxes IA han de completar voltes per verificar que poden córrer en carrera
48	testUpdateLapTime()@Línia337	Comprovar que funciona el fet d'actualitzar o millorar els temps de volta i sectors
49	testPassedAllCheckpoints()@Línia356	Com la mecànica de checkpoints i línia de meta és la mateixa que el jugador, han de passar per tots els checkpoints
50	testNotPassedAllCheckpoints()@Línia367	Mateixa que l'anterior però negada

CarTest.java

nProva	Nom i lloc	Què comprova?
51	GettersSetterTest()@Línia56	Comprovar que s'agafen i afegeixen els valors correctament
52	MovimentTest()@Línia92	Comprovar el moviment amb els inputs corresponents
53	trackLimitsTest()@Línia132	Verificar que es compleix la detecció quan surt de pista
54	testSingleKeyMovement()@Línia165	Comprovar que funciona amb un sol input
55	testMovementWD()@Línia176	Comprovar que funciona amb dos o més inputs
56	testMovementWA()@Línia189	Similar comprovació que l'anterior
57	testMovementSD()@Línia201	Similar comprovació que l'anterior
58	testFrictionMovement()@Línia212	A cada fotograma s'aplica un coeficient de fricció que frena el cotxe molt poc per a reduir velocitat si no es prem res

59	testBoundaryLimit()@Línia219	Test bàsic per comprovar que no se surt del mapa exterior
60	testApplySlipstreamCoverage()@Línia228	Comprova totes les possibilitats de tenir o no Slipstream
61	testApplySlipstreamCoverage2()@Línia239	Comprova totes les possibilitats de tenir o no Slipstream
62	testApplySlipstream3()@Línia250	Comprova totes les possibilitats de tenir o no Slipstream
63	testApplySlipstream4()@Línia261	Comprova totes les possibilitats de tenir o no Slipstream

MapTest.java

(A tenir en compte:

Sector lila = Millor sector personal i de tots els cotxes

Sector verd = Millor sector personal però no de tots els cotxes

Sector taronja = Temps del sector pitjor al millor personal

)

nProva	Nom i lloc	Què comprova?
64	testInitialState()@Línia64	Verificacions d'estat inicial
65	testReset()@Línia75	Verificacions que es pugui reiniciar l'estat
66	testResetSectors()@Línia93	El reset dels Sectors va a part del general per raons de gestió de voltes
67	testRaceInitialState()@Línia106	Comprova l'estat inicial d'una carrera
68	testRaceSettersGetters()@Línia118	Comprovar que s'agafen i afegixen els valors correctament d'una carrera
69	testIniPassedAllCheckpoints()@Línia144	S'inicialitza correctament el flag dels checkpoints
70	testPassedAllCheckpoints()@Línia151	El flag dels checkpoints realitza el seu funcionament per comprovar que ha passat per tots
71	testSum_EmptyArray()@Línia165	Comprovar que no tira excepció la funció per sumar sectors (Afegida més tard)

72	testSum_PartialElements()@Línia173	Comprovar que la funció de sumar sectors funciona correctament
73	testSum_IndexOutOfBounds()@Línia183	Comprovar que no tira excepció la funció per sumar sectors (Afegida més tard)
74	testSector()@Línia193	Els sectors han de funcionar correctament seguint les normes establertes
75	testCheckpointProgression()@Línia221	Verificació del comportament de la progressió de checkpoints
76	testGetBestLapTime@Línia248	Comprovar que s'agafen els valors correctament
77	testGetBestLapTimeFinish()@Línia258	Comprovar que s'agafen els valors correctament
78	testResetSectors()@Línia290	Comprovar que els sectors es reinicien correctament per tornar a iniciar una volta
79	testCopyCurrentToLastSectors()@Línia312	Comprovar que els sectors es copien correctament per imprimir-los per pantalla en acabar una volta
80	testStateTransitions()@Línia328	Comprovar la funcionalitat de tots els estats de jocs
81	testIsRaceComplete()@Línia340	Verificació si ha acabat la carrera o no
82	testIsCountdownActive()@Línia360	Mateix que anterior però amb el countdown d'abans de la carrera
83	testSectorRace()@Línia373	Comprovar que tots els sectors funcionen correctament durant una carrera
84	testFinishRace()@Línia405	Realitzar 3 voltes i acabar una carrera de forma correcta

Proves de caixa negra:

AIControllerTest.java

(No hi ha pràcticament, ja que es necessita principalment la utilització de mock objects, per tant, es troben a l'apartat de proves amb mock objects)

nProva	Nom i lloc	Què comprova?
--------	------------	---------------

1	testUpdatePositions()@Línia140	Verificació funcionament de les posicions i els seus valors possibles
---	--------------------------------	---

CarControllerTest.java

VALORS A MIRAR EN PARTICIONES Y FRONTERA:

TECLES D'ENTRADA: (W | UP) + (S | DOWN) + (A | LEFT) + (D | RIGHT) + NO

VALID

ESTAT DE LA VELOCIDAD (0 - 10, (-5) - 0, 0)

POSICIÓ EN EL MAPA (X=[0, 500] y Y=[0, 500] || x/y < 0 + x/y > 500)

SUPERFÍCIE EN PISTA (EN PISTA, FORA DE PISTA, COL·LISIÓ AMB PARET INVISIBLE)

- COVERAGE ACTUAL 100% DELS VALORS -

TAULA COMPLETA PER COVERAGE:

(A = Accelerar, F = Frenar, I = Esquerra, D = Dreta)

A	F	I	D	Combinació de Tecles
0	0	0	0	Cap tecla
0	0	0	1	D o RIGHT
0	0	1	0	A o LEFT
0	0	1	1	A + D o LEFT + RIGHT
0	1	0	0	S o DOWN
0	1	0	1	S + D o DOWN + RIGHT
0	1	1	0	S + A o DOWN + LEFT
0	1	1	1	S + A + D o DOWN + LEFT + RIGHT
1	0	0	0	W o UP
1	0	0	1	W + D o UP + RIGHT
1	0	1	0	W + A o UP + LEFT
1	0	1	1	W + A + D o UP + LEFT + RIGHT
1	1	0	0	W + S o UP + DOWN
1	1	0	1	W + S + D
1	1	1	0	W + S + A
1	1	1	1	W + S + A + D

nProva	Nom i lloc	Què comprova?
2	testAccelerate()@Línia151	Acceleració del cotxe (W UP)
3	testDecrease()@Línia158	Baixar velocitat del cotxe (S DOWN)
4	testTurnA()@Línia166	Girar el cotxe esquerre (A)
5	testTurnLeft()@Línia174	Girar el cotxe esquerre (LEFT)

6	testTurnD()@Línia182	Girar el cotxe dreta (D)
7	testTurnRight()@Línea190	Girar el cotxe dreta (RIGHT)
8	testInvalidKeyDoesNothing()@Línia198	Una tecla sense bind no fa res
9	testThreeKeysWAD()@Línia208	Pairwise W + A + D
10	testThreeKeysSAD()@Línia219	Pairwise S + A + D
11	testOppositeKeysWA_SD()@Línia229	Pairwise W + S && A + D
12	testUpdateMovesCar()@Línia240	Estat de velocitat
13	testVelocityMax()@Línia248	Estat de velocitat màx
14	testVelocityBackwardsMax()@Línia256	Estat de velocitat min
15	testPairwiseWA()@Línia263	Pairwise W + A
16	testPairwiseWD()@Línia270	Pairwise W + D
17	testPairwiseSA()@Línia277	Pairwise S + A
18	testPairwiseSD()@Línia284	Pairwise S + D
19	testEdgeMaxVelocity()@Línia291	Valors Frontera Velocitat Màx
20	testEdgeMinVelocity()@Línia299	Valors Frontera Velocitat Min
21	testEdgeJustMaxVelocity()@Línia307	Valors veïns velocitat màx per sota
22	testEdgeJustMinVelocity()@Línia314	Valors veïns velocitat min per sota
23	testEdgeAboveMaxVelocity()@Línia321	Valors veïns velocitat màx per sobre
24	testEdgeAboveMinVelocity()@Línia328	Valors veïns velocitat min per sobre
25	testMovementJustRightBoundary()@Línia336	Valors veïns mapa dreta per sota
26	testMovementAboveRightBoundary()@Línia344	Valors veïns mapa dreta per sobre
27	testMovementJustLeftBoundary()@Línia352	Valors veïns mapa esquerra per sota
28	testMovementAboveLeftBoundary()@Línia360	Valors veïns mapa esquerra per sobre

MapControllerTest.java

VALORS A MIRAR EN PARTICIONS I FRONTERA:

ESTATS DEL MAPA: IDLE, RUNNING, OFF_TRACK, INVALID_MAP, RESULT

LIMITS EN LÍNIA DE META I CHECKPOINTS

TIMERS

SECTORS 1,2,3 - COLORS NONE, ORANGE, GREEN, PURPLE

- COVERAGE ACTUAL 100% DELS VALORS -

nProva	Nom i lloc	Què comprova?
29	testIdleToRunningTransition()@Línia478	IDLE → RUNNING
30	testRunningToCompletedTransition@Línia486	RUNNING → COMPLETED
31	testRunningToOffTrackTransition@Línia498	RUNNING → OFFTRACK
32	testOffTrackAnyPosition()@Línia507	Manté estat OFFTRACK sense canviar
33	testNoOffTrackFinishLineAllCheckpoints@Línia528	NO OFFTRACK + RUNNING + FINISH LINE = RESULT
34	testNoOffTrackCheckpointPosition@Línia542	NO OFFTRACK + CHECKPOINT = RUNNING
35	testRegularPosition()@Línia554	NO OFFTRACK = RUNNING
36	testOffTrackNotFinishLine()@Línia566	OFFTRACK + Moviment sense importància en la traçada = OFFTRACK
37	testFinishLineBoundaryInside()@Línia577	Frontera i veïns interns del contacte amb la línia de meta
38	testFinishLineBoundaryOutside()@Línia617	Frontera i veïns externs del contacte amb la línia de meta
39	testCheckpointBoundaryInside()@Línia638	Frontera i veïns interns del contacte amb un checkpoint
40	testCheckpointBoundaryOutside()@Línia692	Frontera i veïns externs del contacte amb un checkpoint
41	testFinishLineOffTrackPairwise()@Línia731	En Meta + OFFTRACK en meta = OFFTRACK
42	testIdleStartsLap()@Línia739	IDLE + En meta + NO OFFTRACK = Comença volta
43	testRunningAtFinishLineCompletesLap()@Línia747	RUNNING + En meta + tots checkpoints = Acaba volta
44	testIdleOffTrackPairwise()@Línia761	IDLE + Fora de meta + OFFTRACK = OFFTRACK
45	testCheckpointOffTrackPairwise()@Línia769	Checkpoint + OFFTRACK = OFFTRACK sense marcar checkpoint
46	testOffTrackStartsPairwise()@Línia779	OFFTRACK + Finish Line = Comença volta (diferent del 42)
47	testSectorPartitions()@Línia790	Particions de tots els tipus de sectors

AlCarTest.java

VALORS A MIRAR EN PARTICIONS I FRONTERA:

SKILL LEVEL DEL COTXES
SLIPSTREAM BOUNDARIES (DISTANCE I LAP)
MAP BOUNDARIES
NORMALITZACIÓ D'ANGLES

- COVERAGE ACTUAL 100% DELS VALORS -

nProva	Nom i lloc	Què comprova?
48	testSkillLevelBoundaries()@Línia411	Particions, valors fronters, veïns del nivell de destresa dels cotxes
49	testSlipstreamDistanceBoundaries()@Línia431	Particions, valors fronters, veïns de la distància de Slipstream
50	testSlipstreamLapBoundaries()@Línia454	Particions, valors fronters, veïns de la volta on s'aplica Slipstream (a partir de la segona)
51	testMapBoundaryBehavior()@Línia471	Valors fronters i veïns del mapa
52	testNormalizeAngle()@Línia484	Particions, valors fronters, veïns possibles d'un angle dins la partida

CarTest.java

VALORS A MIRAR EN PARTICIONS I FRONTERA:

TECLES D'ENTRADA: W | UP + S | DOWN + A | LEFT + D | RIGHT + NO VALID
ESTADO DE LA VELOCIDAD (0-10, -5-0, 0)
POSICIÓN EN EL MAPA (X=[0, 500] y Y=[0, 500] || x/y < 0 + x/y > 500)
SUPERFICIE EN PISTA (EN PISTA, FUERA DE PISTA, COLISIÓN CON PARED
INVISIBLE)

- COVERAGE ACTUAL 100% DE LOS VALORES -

PAIRWISE TABLE:

(A = Accelerar, F = Frenar, I = Esquerra, D = Dreta)

A F	I	D	Combinació	Estat Velocitat	Angle
0 0	0	0	Cap	Zero	Cualquiera
0 0	0	1	Solo D	Zero/Positiva	Aumenta
0 0	1	0	Solo A	Zero/Positiva	Disminuye
0 0	1	1	A + D	Zero/Positiva	Neutral
0 1	0	0	Solo S	Positiva/Negativa	Qualsevol
0 1	0	1	S + D	Negativa	Augmenta

0 1	1	0	S + A	Negativa	Disminueix
0 1	1	1	S + A + D	Negativa	Neutral
1 0	0	0	Solo W	Zero/Positiva	Cualquiera
1 0	0	1	W + D	Positiva	Augmenta
1 0	1	0	W + A	Positiva	Disminueix
1 0	1	1	W + A + D	Positiva	Neutral
1 1	0	0	W + S	Positiva/Negativa	Qualsevol

nProva	Nom i lloc	Què comprova?
53	settersGettersPartitionTest()@Línia305	Valors possibles de x, y, angle i velocitat als getters i setters (tenen comprovació)
54	forwardMovementPartitionTest()@Línia352	Valors possibles per avançar el cotxe
55	backwardsMovementPartitionTest()@Línia371	Valors possibles per fer retrocedir el cotxe
56	LeftMovementPartitionTest()@Línia390	Valors possibles per fer girar el cotxe cap a l'esquerra
57	RightMovementPartitionTest()@Línia409	Valors possibles per fer girar el cotxe cap a la dreta
58	NoMovementKeyPartitionTest()@Línia428	Una altra tecla no fa res
59	trackLimitsPartitionTest()@Línia437	Particions, valors fronters i veïns sobre trackLimits
60	inviWallPartitionTest()@Línia517	Particions, valors fronters i veïns sobre col·lisions amb parets
61	updatePartitionTest()@Línia562	Valors possibles de velocitat després d'un update
62	testMovementVelocityBoundaries()@Línia602	Mirar que el cotxe no pugui passar de velocitat màx i min
63	testVelocitySignChange()@Línia615	Mirar que el cotxe pugui tirar marxa enrere amb velocitat negativa
64	testPairwiseMovement()@Línia624	Provar totes les combinacions possibles de moviment de la taula de dalt
65	testEdgeVelocityBoundary()@Línia707	Valors fronters i veïns sobre la velocitat del cotxe, cap a dalt i baix
66	testEdgeMapBoundaries()@Línia736	Valors fronters i veïns sobre la posició del cotxe al mapa, cap a qualsevol direcció
67	testEdgeExtremeValues()@Línia788	Valors extrems per a la posició en el mapa
68	testEdgeFriction()@Línia807	Valors molt propers a 0 es controlen

Proves de Caixa Blanca:

(A decision coverage, condition coverage, path coverage i loop testing, realment existeixen més exemples dels posats a l'informe, tal com es pot veure a la captura del statement coverage on no hi ha cap branca sense cobrir més que les de la vista del joc, ja que més d'una necessita proves "Headless" i més complexes.

Statement Coverage

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 es.uab.tqs.f1_2D.vista		70 %		61 %	46	153	155	574	3	64	0	7
 es.uab.tqs.f1_2D.controlador		100 %		100 %	0	137	0	389	0	57	0	4
 es.uab.tqs.f1_2D.model		100 %		100 %	0	199	0	409	0	113	0	5
Total	804 of 7.118	88 %	68 of 507	86 %	46	489	155	1.372	3	234	0	16

Decision Coverage:

handleCheckpoints()

```
132. //Función para comprobar si el coche se encuentra encima de un checkpoint
133. private void handleCheckpoints(double x, double y)
134. {
135.     var checkpoints = model.getCheckpoints();
136.     //Iterar sobre la lista de checkpoints que contiene el circuito actual
137.     for (int i = 0; i < checkpoints.size(); i++)
138.     {
139.         if (checkpoints.get(i).contains(x + 40, y + 40))
140.         {
141.             //Si se encuentra encima de un checkpoint donde ya ha pasado no hace nada
142.             if (model.getPassedCheckpoints().contains(i)) break;
143.
144.             //Si el checkpoint es el siguiente que debe visitar por orden de circuito, graba sector y tiempo
145.             //En caso contrario invalida vuelta ya que no está siguiendo el circuito correctamente
146.             if (i == model.getNextCheckpointIndex())
147.             {
148.                 model.getPassedCheckpoints().add(i);
149.                 model.setNextCheckpointIndex(model.getNextCheckpointIndex() + 1);
150.
151.                 // Grabar sector (excepto para el primer y último checkpoint)
152.                 if (i >= 1 && i <= checkpoints.size() - 2)
153.                 {
154.                     int sectorIndex = i - 1;
155.                     long now = timeProvider.now();
156.                     long totalSinceStart = now - model.getLapStartTime();
157.                     long prevSum = model.sum(model.getSectorTimes(), sectorIndex);
158.                     long sectorTime = totalSinceStart - prevSum;
159.                     recordSector(sectorIndex, sectorTime);
160.                 }
161.             }
162.             else
163.             {
164.                 handleInvalidLap();
165.                 break;
166.             }
167.         }
168.     }
169. }
```

handleFinishLineCrossing()

```
89. //Si el coche se encuentra encima de la línea de meta, controla su estado
90. private void handleFinishLineCrossing()
91. {
92.     Map.State currentState = model.getState();
93.
94.     //Si viene de hacer una vuelta invalida, resetea su estado
95.     if (currentState == Map.State.OFF_TRACK || currentState == Map.State.INVALID_LAP)
96.     {
97.         model.setCurrentState(Map.State.IDLE);
98.     }
99.
100.    //Si su estado es el default, empieza vuelta
101.    if (currentState == Map.State.IDLE)
102.    {
103.        startLap();
104.    }
105.    else
106.    {
107.        //Si está corriendo mira si es el último checkpoint y cuenta el último sector
108.        int lastSectorIndex = model.getNumSectors() - 1;
109.        if (!model.getSectorRecorded()[lastSectorIndex])
110.        {
111.            if(model.getNextCheckpointIndex() != 0)
112.            {
113.                long now = timeProvider.now();
114.                long totalSinceStart = now - model.getLapStartTime();
115.                long prevSum = model.sum(model.getSectorTimes(), lastSectorIndex);
116.                long sectorTime = totalSinceStart - prevSum;
117.                recordSector(lastSectorIndex, sectorTime);
118.            }
119.        }
120.
121.        //Si ya ha pasado por todos los checkpoints acaba vuelta, en caso contrario la vuelve a empezar
122.        if (model.passedAllCheckpoints())
123.        {
124.            endLap();
125.        } else
126.        {
127.            startLap();
128.        }
129.    }
130. }
```

applySlipstream()

```
88. //Aplicar Slipstream a los coches rivales
89. public void applySlipstream(List<AICar> otherAICars, int currentLap)
90. {
91.     slipstreamBoost = 0;
92.
93.     //Solo aplicar a partir de la segunda vuelta
94.     if (currentLap >= 2)
95.     {
96.         // Buscar coches cercanos por delante
97.         for (AICar otherCar : otherAICars)
98.         {
99.             //Si el coche se encuentra justo detrás
100.            if (isBehind(otherCar))
101.            {
102.                //Calcular la distancia hasta el coche
103.                double distance = calculateDistance(otherCar);
104.
105.                //Si la distancia es menor a 100 pixeles, aplicar slipstream
106.                if (distance < 100)
107.                {
108.                    slipstreamBoost = 5.0 * skillLevel;
109.                }
110.            }
111.        }
112.
113.        //Si se encuentra detrás del jugador
114.        if (isBehindPlayer())
115.        {
116.            //Calcular la distancia al jugador
117.            double distance = calculateDistanceToPlayer();
118.
119.            //Si la distancia es menor a 100 aplicar slipstream
120.            if (distance < 100)
121.            {
122.                slipstreamBoost = Math.max(slipstreamBoost, 5.0 * skillLevel);
123.            }
124.        }
125.    }
126. }
```


Condition Coverage:

movement()

```
44. //Función para controlar el movimiento en base a las teclas
45. public Boolean movement(Set<Integer> keys)
46. {
47.     boolean moved = false;
48.     //Calcular maxVelocity + el boost del slipstream
49.     double effectiveMaxVelocity = getEffectiveMaxVelocity();
50.
51.     //Si contiene teclas de avance, aumentar la velocidad o mantener en caso de máximos
52.     ◆ if (keys.contains(KeyEvent.VK_W) || keys.contains(KeyEvent.VK_UP)) {
53.         velocity += acceleration;
54.         ◆ if (velocity > effectiveMaxVelocity) velocity = effectiveMaxVelocity;
55.         moved = true;
56.     }
57.
58.     //Si contiene teclas de retroceso/freno, disminuir la velocidad o mantener en caso de máximos
59.     ◆ if (keys.contains(KeyEvent.VK_S) || keys.contains(KeyEvent.VK_DOWN)) {
60.         velocity -= backwardsAcceleration;
61.         ◆ if (velocity < backwardsMaxVelocity) velocity = backwardsMaxVelocity;
62.         moved = true;
63.     }
64.
65.     //Si contiene teclas de rotación hacia la izquierda, disminuir el angulo
66.     ◆ if (keys.contains(KeyEvent.VK_A) || keys.contains(KeyEvent.VK_LEFT)) {
67.         angle -= 5;
68.         moved = true;
69.     }
70.
71.     //Si contiene teclas de rotación hacia la derecha, disminuir el angulo
72.     ◆ if (keys.contains(KeyEvent.VK_D) || keys.contains(KeyEvent.VK_RIGHT)) {
73.         angle += 5;
74.         moved = true;
75.     }
76.
77.     return moved;
78. }
```

stopAllTimers()

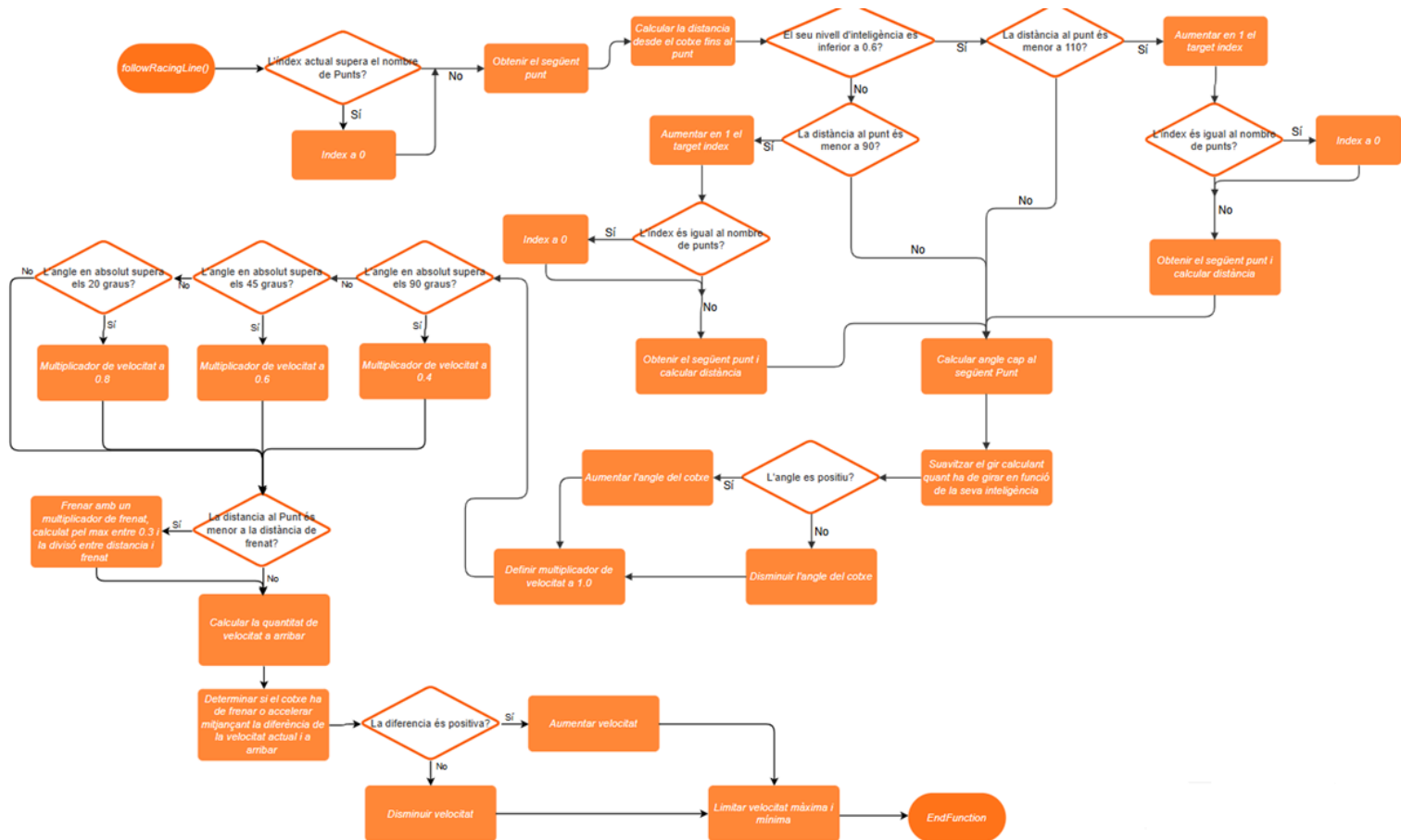
```
369. //Función para parar todos los timers y evitar descontrol de variables sobrescritas
370. public void stopAllTimers()
371. {
372.     ◆ if (resultTimer != null && resultTimer.isRunning())
373.     {
374.         resultTimer.stop();
375.     }
376.     ◆ if (offTrackTimer != null && offTrackTimer.isRunning())
377.     {
378.         offTrackTimer.stop();
379.     }
380.     ◆ if (countdownTimer != null && countdownTimer.isRunning())
381.     {
382.         countdownTimer.stop();
383.     }
384. }
```

recordSector()

```
386.    //Si se detecta que se ha finalizado un sector,
387.    //se cuenta el tiempo y se determina qué color es dependiendo del tiempo
388.    public void recordSector(int sectorIndex, long sectorTime)
389.    {
390.        //Comprobar que el sector existe
391.        if (sectorIndex < 0 || sectorIndex >= model.getNumSectors()) return;
392.
393.        //Asignar el tiempo establecido
394.        model.setSectorTime(sectorIndex, sectorTime);
395.        model.setSectorRecorded(sectorIndex, true);
396.
397.        //Obtener los mejores tiempos de sector
398.        long previousLocalBest = model.getBestSectorTime(sectorIndex);
399.
400.        //Si es una mejora local, determinar verde, en caso contrario naranja
401.        if (sectorTime < previousLocalBest)
402.        {
403.            model.setBestSectorTime(sectorIndex, sectorTime);
404.            model.setSectorColor(sectorIndex, Map.SectorColor.GREEN);
405.        }
406.        else
407.        {
408.            model.setSectorColor(sectorIndex, Map.SectorColor.ORANGE);
409.            return;
410.        }
411.
412.        long min = Long.MAX_VALUE;
413.        if(model.isRaceMode())
414.        {
415.            //Si estamos en carrera, comprobar quien tiene el mejor tiempo de ese sector
416.            //si se trata del jugador, poner el sector en morado
417.            for(AICar aicar : aiController.getAICars())
418.            {
419.                Long aiTime = aicar.getSectorTimes()[sectorIndex];
420.                if(aiTime < min)
421.                {
422.                    min = aiTime;
423.                }
424.            }
425.
426.            if(min > sectorTime)
427.            {
428.                model.setSectorColor(sectorIndex, Map.SectorColor.PURPLE);
429.            }
430.        }
431.    }
```

PathCoverage:

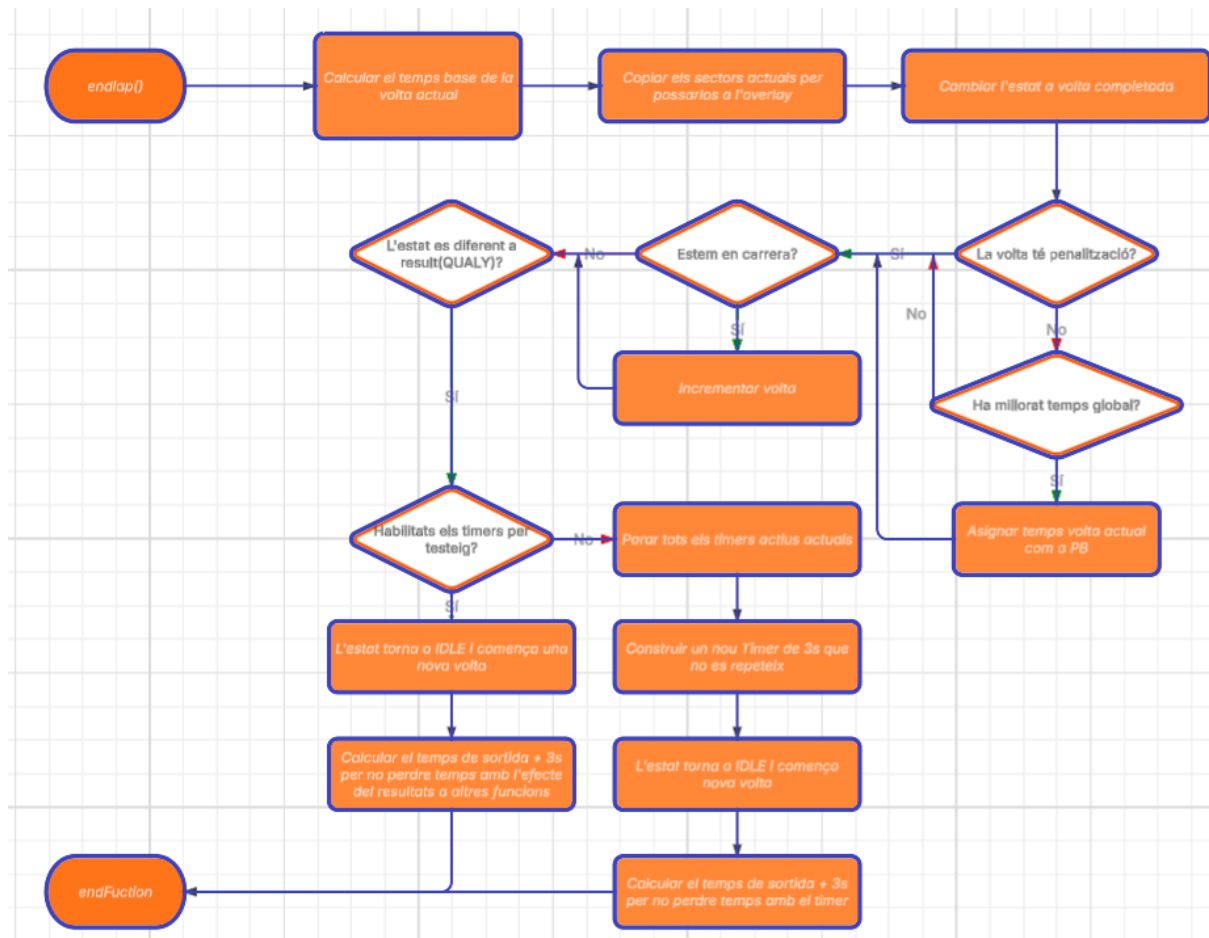
followRacingLine()



```
128. //Función principal para seguir la Racing Line (no al pie de la letra) para saber por donde ir
129. public void followRacingLine()
130. {
131.     //Resetear indice para el principio de vueltas
132.     if (currentTargetIndex >= racingLine.size())
133.     {
134.         currentTargetIndex = 0;
135.     }
136.
137.     //Obtener el siguiente indice que debe seguir el coche
138.     Point target = racingLine.get(currentTargetIndex);
139.     double targetX = target.x;
140.     double targetY = target.y;
141.
142.     double dx = targetX - x;
143.     double dy = targetY - y;
144.
145.     //Calcular la distancia hasta el siguiente punto
146.     double distanceToTarget = Math.sqrt(dx * dx + dy * dy);
147.
148.     //Si el skill level es inferior, mirar más adelante los checkpoints para evitar chocarse
149.     if(skillLevel < 0.6)
150.     {
151.         //Si estamos muy cerca del waypoint, pasar al siguiente
152.         if (distanceToTarget < 110)
153.         {
154.             currentTargetIndex++;
155.
156.             //Tener en cuenta que el siguiente puede ser la línea de meta
157.             if(currentTargetIndex == racingLine.size()) currentTargetIndex = 0;
158.
159.             //Como se ha cambiado el siguiente punto, cambiar rumbo y calcular distancia
160.             target = racingLine.get(currentTargetIndex);
161.             targetX = target.x;
162.             targetY = target.y;
163.
164.             dx = targetX - x;
165.             dy = targetY - y;
166.             distanceToTarget = Math.sqrt(dx * dx + dy * dy);
167.
168.         }
169.     }
170.     //Si tienen un skill level más alto, no les hace falta tanta distancia de detección
171.     else
172.     {
173.         //Si estamos muy cerca del waypoint, pasar al siguiente
174.         if (distanceToTarget < 90)
175.         {
176.             currentTargetIndex++;
177.
178.             //Tener en cuenta que el siguiente puede ser la línea de meta
179.             if(currentTargetIndex == racingLine.size()) currentTargetIndex = 0;
180.         }
181.     }
182. }
```

```
180.
181.         //Como se ha cambiado el siguiente punto, cambiar rumbo y calcular distancia
182.         target = racingLine.get(currentTargetIndex);
183.         targetX = target.x;
184.         targetY = target.y;
185.
186.         dx = targetX - x;
187.         dy = targetY - y;
188.         distanceToTarget = Math.sqrt(dx * dx + dy * dy);
189.
190.     }
191. }
192.
193. //Calcular ángulo hacia el siguiente objetivo
194. double targetAngle = Math.toDegrees(Math.atan2(dy, dx));
195.
196. //Suavizar el giro en función del skill level (más giro o menos giro)
197. double angleDiff = normalizeAngle(targetAngle - angle);
198. double turnAmount = Math.min(5.0 * skillLevel, Math.abs(angleDiff));
199.
200. //Decidir dirección de giro
201. if (angleDiff > 0)
202. {
203.     angle += turnAmount;
204. }
205. else
206. {
207.     angle -= turnAmount;
208. }
209.
210. //Control de velocidad
211. double speedMultiplier = 1.0;
212.
213. //Reducir velocidad en curvas pronunciadas
214. if (Math.abs(angleDiff) > 90)
215. {
216.     speedMultiplier = 0.4;
217. }
218. else if (Math.abs(angleDiff) > 45)
219. {
220.     speedMultiplier = 0.6;
221. }
222. else if (Math.abs(angleDiff) > 20)
223. {
224.     speedMultiplier = 0.8;
225. }
226.
227. //Reducir velocidad al acercarse al waypoint
228. if (distanceToTarget < brakingDistance)
229. {
230.     double brakeMultiplier = Math.max(0.3, distanceToTarget / brakingDistance);
231.     speedMultiplier *= brakeMultiplier;
232. }
233.
234. double currentTargetSpeed = (targetSpeed + slipstreamBoost) * speedMultiplier;
235.
236. //Determinar si el coche debe frenar o acelerar
237. double speedDifference = currentTargetSpeed - velocity;
238.
239. //Decidir si el coche debe frenar o acelerar según su situación
240. if (speedDifference > 0)
241. {
242.
243.     velocity += acceleration * skillLevel * Math.min(1.0, speedDifference / 5.0);
244. }
245. else
246. {
247.
248.     velocity += backwardsAcceleration * skillLevel * Math.max(-1.0, speedDifference / 5.0);
249. }
250.
251. // Limitar velocidad máxima y mínima
252. velocity = Math.max(backwardsMaxVelocity, Math.min(getEffectiveMaxVelocity(), velocity));
253. }
```


endLap()



```
247. //Si se ha pasado por línea de meta y la vuelta se da por finalizada
248. //se muestra el resultado o se incrementa vuelta en RACE
249. private void endLap()
250. {
251.     //Calcular el tiempo base de la vuelta
252.     long baseLapTime = timeProvider.now() - model.getLapStartTime();
253.
254.     //Copiar los arrays de sectores para pasarlos al overlay
255.     model.copyCurrentToLastSectors();
256.
257.     //Cambiar estado a vuelta completada
258.     model.setCurrentState(Map.State.RESULT);
259.
260.     //Si no tiene castigos y es su mejor vuelta, asignarlo
261.     if (!model.isLapHasPenalty())
262.     {
263.         if (baseLapTime < model.getBestLapTimeFinish())
264.             model.setBestLapTime(baseLapTime);
265.     }
266.
267.     //Si estamos en carrera, incrementar vuelta
268.     if (model.isRaceMode())
269.     {
270.         model.incrementLap();
271.     }
272.
273.     //Si la carrera no se ha acabado o estamos en modo qualy
274.     if (model.getState() != Map.State.RACE_FINISHED)
275.     {
276.         //Branch para poder testear sin timers
277.         if (!isSkipEnabled)
278.         {
279.             //Enseñar los resultados de la vuelta y preparar la siguiente
280.             stopAllTimers();
281.             resultTimer = new Timer(3000, e ->
282.             {
283.                 model.setCurrentState(Map.State.IDLE);
284.                 startLap();
285.                 model.setLapStartTime(timeProvider.now() - 3000);
286.                 resultTimer.stop();
287.             });
288.             resultTimer.setRepeats(false);
289.             resultTimer.start();
290.         }
291.         else
292.         {
293.             //Empezar siguiente vuelta sin enseñar los resultados, solo para testing
294.             model.setCurrentState(Map.State.IDLE);
295.             startLap();
296.             model.setLapStartTime(timeProvider.now() - 3000);
297.         }
298.     }
299. }
```


LoopTesting:

Aniats:

loadAISprites()

```
876. //Per cada cotxe, assignar-li el seu Sprite corresponent al nom de l'equip
877. for (AICar aiCar : aiController.getAICars())
878. {
879.     String team = aiCar.getTeam().toLowerCase().replace(" ", "");
880.     int teamIndex = -1;
881.
882.     //trobar l'equip dins de l'array
883.     for (int i = 0; i < teamNames.length; i++)
884.     {
885.         if (teamNames[i].equals(team))
886.         {
887.             teamIndex = i;
888.             break;
889.         }
890.     }
891.
892.     // Si no es troba, utilitzar index per defecte
893.     if (teamIndex == -1)
894.     {
895.         teamIndex = 0;
896.     }
897.
898.     //Carregar la direcció en la funció principal de load
899.     loadCarSprite(aiCar, teamColors[teamIndex]);
900. }
901. }
```

generateRacingLine()

```
141. // Añadir waypoints intermedios para mejor navegación
142. for (int i = 0; i < keyPoints.length - 1; i++) {
143.     Point current = keyPoints[i];
144.     Point next = keyPoints[i + 1];
145.
146.     // Añadir punto actual
147.     racingLine.add(current);
148.
149.     // Añadir puntos intermedios si la distancia es grande
150.     double distance = Math.sqrt(Math.pow(next.x - current.x, 2) + Math.pow(next.y - current.y, 2));
151.     //Si la distancia es mayor a 150 píxeles añadir uno cada 75 píxeles
152.     if (distance > 150)
153.     {
154.         int intermediatePoints = (int)(distance / 75);
155.         for (int j = 1; j < intermediatePoints; j++) {
156.             double t = (double) j / intermediatePoints;
157.             int interX = (int) (current.x + (next.x - current.x) * t);
158.             int interY = (int) (current.y + (next.y - current.y) * t);
159.             racingLine.add(new Point(interX, interY));
160.         }
161.     }
162. }
```

Simples:

updateAICheckpoints()

```
272. //Verificar si se encuentra en la finish line o encima de un checkpoint
273. private void updateAICheckpoints(AICar aiCar)
274. {
275.     //Verificar línea de meta
276.     if (trackMap.getFinishLine().contains((int)(aiCar.getX() + 40), (int)(aiCar.getY() + 40)))
277.     {
278.         handleAIFinishLine(aiCar);
279.         return;
280.     }
281.
282.     //Verificar checkpoints normales
283.     var checkpoints = trackMap.getCheckpoints();
284.     for (int i = 0; i < checkpoints.size(); i++)
285.     {
286.         Rectangle checkpoint = checkpoints.get(i);
287.         if (checkpoint.contains((int)(aiCar.getX() + 40), (int)(aiCar.getY() + 40)))
288.         {
289.             handleAICheckpoint(aiCar, i);
290.             break;
291.         }
292.     }
293. }
```

sum()

```
91. //Función que suma los tiempos de los sectores
92. public long sum(long[] array, int index)
93. {
94.     if (index <= 0) return 0L;
95.     long suma = 0L;
96.     for (int i = 0; i < index && i < array.length; i++) suma += array[i];
97.     return suma;
98. }
```

Automatization Data Driven:

AlCarTest.java

```
//Lista de argumentos para tests de parametrización
static Stream<Arguments> skillLevelProvider()
{
    return Stream.of(
        //Skill Level Bajo [0.1 - 0.3]
        Arguments.of(0.1, 0.7, 120.0),
        Arguments.of(0.2, 0.72, 115.0),
        Arguments.of(0.3, 0.74, 110.0),

        //Skill Level Medio [0.4 - 0.6]
        Arguments.of(0.4, 0.76, 105.0),
        Arguments.of(0.5, 0.78, 100.0),
        Arguments.of(0.6, 0.8, 95.0),

        //Skill Level Alto [0.7 - 1.0]
        Arguments.of(0.7, 0.82, 90.0),
        Arguments.of(0.85, 0.87, 85.0),
        Arguments.of(1.0, 0.9, 80.0)
    );
}

static Stream<Arguments> slipstreamScenarioProvider()
{
    return Stream.of(
        //Sin slipstream
        Arguments.of(1, 100.0, 0.0, "No slipstream en vuelta 1"),

        //Slipstream posible
        Arguments.of(2, 200.0, 0.0, "Slipstream posible pero sin objetivo"),

        //Slipstream activo
        Arguments.of(3, 50.0, 5.0, "Slipstream activo con coche cercano")
    );
}
```

```
66 static Stream<Arguments> pairwiseParameters()  
67 {  
68     return Stream.of(  
69         // Combinaciones pairwise para skill level, posición, y estado de carrera  
70         Arguments.of(0.2, 100.0, 100.0, 1, "Baja skill, posición normal, vuelta 1"),  
71         Arguments.of(0.2, 0.0, 0.0, 3, "Baja skill, esquina, vuelta 3"),  
72         Arguments.of(0.8, 100.0, 0.0, 1, "Alta skill, borde superior, vuelta 1"),  
73         Arguments.of(0.8, 0.0, 100.0, 3, "Alta skill, borde inferior, vuelta 3"),  
74         Arguments.of(0.5, 500.0, 500.0, 2, "Skill media, centro, vuelta 2"),  
75         Arguments.of(0.5, 0.0, 0.0, 1, "Skill media, esquina, vuelta 1"),  
76         Arguments.of(1.0, 100.0, 100.0, 3, "Máxima skill, normal, vuelta 3"),  
77         Arguments.of(0.1, 999.0, 999.0, 2, "Mínima skill, esquina opuesta, vuelta 2")  
78     );  
79 }  
80  
81 static Stream<Arguments> teamAndSkillPairwise() {  
82     return Stream.of(  
83         Arguments.of("Mercedes", 0.9, "Equipo top, alta skill"),  
84         Arguments.of("Mercedes", 0.3, "Equipo top, baja skill"),  
85         Arguments.of("Haas", 0.9, "Equipo bajo, alta skill"),  
86         Arguments.of("Haas", 0.3, "Equipo bajo, baja skill"),  
87         Arguments.of("Ferrari", 0.6, "Equipo medio, skill media"),  
88         Arguments.of("Williams", 0.7, "Equipo medio-bajo, skill media-alta")  
89     );  
90 }
```

nProva	Nom i lloc	Què comprova?
1	testAICarSkillLevel()@Línia384	Automatització per comprovar skill levels
2	testSlipstream()@Línia393	Diferents escenaris per aplicar o no, slipstream
3	testPairwiseCombinations()@Línia516	Diferents paràmetres pels cotxes IA
4	testTeamAndSkillCombinations()@Línia536	Diferents combinacions d'equips i de habilitats

MapTest.java

nProva	Nom i lloc	Què comprova?
1	testGetSectorTimeInvalid()@Línia266	Diferents valors de temps pels sectors
2	testGetSectorTimeValid()@Línia274	Diferents valors de temps pels sectors
3	testGetBestSectorTimeInvalid()@Línia283	Diferents valors de millors temps pels sectors

Mock Objects:

MockObjects amb classe apart:

- RandomGenerator.java
- TimeProvider.java

MockObjects amb mockito:

- AIController.java
- AICar.java
- CarController.java
- MapController.java
- Car.java
- Map.java
- BufferedImage per la colisió del mapa
- Rectangle para checkpoints y finish line

Tests amb Mocks Involucrats:

AIControllerTest.java

nProva	Nom i lloc	Què comprova?
1	testUpdateAllAIWithMocksWithCheckpoint()@Línia162	Funcionament de la funció UpdateAllAI per a que es puguin moure els cotxes
2	testUpdateAllAIWithMocksWithout()@Línia188	Funcionament de la funció UpdateAllAI per a que no falli si es surten de pista
3	testHandleFinishLineStartLap()@Línia214	Verificar que poden passar per línia de meta i començar una volta
4	testHandleFinishLineAllCheckpointsPassed()@Línia229	Al passar per tots els checkpoints i després per línia de meta, han de poder acabar una volta
5	testHandleAIFinishLine_NotPassedAllCheckpoints()@Línia242	Si no passa per tots els checkpoints, no pot acabar una volta
6	testHandleAIFinishLineCalled()@Línia255	Comprovar la funcionalitat dels cotxes AI per passar per els checkpoints amb diferents valors al igual que per línia de meta
7	testUpdatePositionsPlayerIA()@Línia274	Verificar que el sort dels cotxes per les posicions en temps reals funciona

		correctament
8	testCheckpointAlreadyPassed()@Línia304	Mirar que si ja ha passat per un checkpoint no es compti de nou
9	testCheckpointRecordSector()@Línia315	Al completar un sector s'ha de registrar el temps d'aquest
10	testCheckpointtOutsideSectorRange()@Línia332	Nomès s'han de comptar el nombre de sectors existents
11	testCheckpointIncorrectReset()@Línia343	Mirar que es reseteixin els checkpoints correctament
12	testCheckpointIncorrect()@Línia357	Mirar que es seleccioni correctament un checkpoint passat per un cotxe IA
13	testHandleAICheckpointSectors()@Línia373	Verificar que es suma correctament els temps dels sectors marcat per IA
14	testGetDistanceToNextCheckpoint()@Línia396	Verificació que major nombre de checkpoints passa al següent
15	testUpdateGlobalSectorTime()@Línia410	Verificació sectors s'assignen correctament al seu color

CarControllerTest.java

nProva	Nom i lloc	Què comprova?
1	testMockMovement()@Línia374	Provar que s'invoca correctament el moviment i es fa l'update al prèmer un input
2	testMockMovementInvalid()@Línia385	Provar que s'invoca correctament el moviment i es fa l'update al prèmer quasevol tipus d'input invàlid
3	testMockMultipleValidKeys()@Línia405	Provar que s'invoca correctament el moviment i es fa l'update al prèmer més d'un input a la vegada
4	testMockMixedValidInvalidKeys()@Línia416	Provar que s'invoca correctament el moviment i es fa l'update al prèmer quasevol tipus d'input invàlid amb un vàlid

MapControllerTest.java

nProva	Nom i lloc	Què comprova?
1	testUpdatePositionMock()@Línia826	Verificar que es detecta la línia de meta i canvia d'estat
2	testCheckpointMock()@Línia844	Verificar que es detecta els checkpoints i s'agreguen a la llista
3	testFinishLineMock()@Línia866	Verificar que es comença la volta correctament
4	testCarMock()@Línia884	Verificar que el cotxe es pot moure desde el controller
5	testWorseBestLapTime()@Línia902	Verificar que en acabar una volta es guarda el temps, en aquest cas, pitjor que l'anterior, per tant, no es guarda
6	testBestLapTime()@Línia933	Verificar que en acabar una volta es guarda el temps, en aquest cas, millor que l'anterior pel qual es guarda
7	testStartRaceMode()@Línia973	Verificar que en començar una carrera s'inicialitza correctament
8	testUpdatePositionDuringCountdown()@Línia988	Verificar que durant el compte enrere no s'actualitza res
9	testCountdownEndsAndRaceStarts()@Línia1002	Quan acaba el compte enrere, la carrera ha de començar
10	testIncrementLapInRaceMode()@Línia1018	Verificar que s'incrementa una volta
11	testRaceCompletion()@Línia1042	Verificar que al cap de 3 voltes es completa la carrera
12	testQualyModeDoesNotIncrementLap()@Línia1067	Verificar que si estem a Qualy, no es compten les voltes però sí els temps
13	testStopAllTimersVariousTimerStates()@Línia1085	Verificar els diferents estats dels timers
14	testInvalidLapRaceMode()@Línia1120	Verificar que el mode carrera no invalida la volta al complet
15	testAfterInvalidLapReturnsToRunningInRaceMode()@Línia1137	Verificar que en invalidar s'ha de tornar a running amb la penalització corresponent en carrera
16	testGetCountdownTimer()@Línia1159	Verificar que el CountDown timer funciona correctament

AICarTest.java

nProva	Nom i lloc	Què comprova?
17	testAICarUpdateWithMockCollision()@Línia555	Verificar que els cotxes IA també tenen col·lisions amb el mapa

CarTest.java

nProva	Nom i lloc	Què comprova?
18	testTrackLimitsFalse()@Línia824	Verificar que no es detecta tracklimits amb el color de pista
19	testTrackLimitsTrue()@Línia834	Verificar que es detecta tracklimits amb el color de l'herba
20	testTrackLimitsWall()@Línia844	Verificar que es para en estar en tracklimits i trobar una paret
21	testInviWallFalse()@Línia857	Verificar que no es detecta paret amb el color de la pista
22	testInviWallTrue()@Línia867	Verificar que es detecta una paret quan el píxel on es troba el cotxe es de color blau (mirar collisionMap.png)
23	testInviWallNoMovement()@Línia877	Verificar que en estar en una paret no es pot moure per aquell espai

MapTest.java

nProva	Nom i lloc	Què comprova?
24	testStartCountdown()@Línia427	Verificar que el countdown inicialitza el seu estat i el conjunt de la carrera
25	testGetRemainingCountdownWhenActive()@Línia440	Verificar que el count del countdown funciona correctament
26	testStartRace()@Línia472	Verificar que el jugador pot iniciar una carrera amb totes les variables inicialitzades correctament
27	testIncrementLap()@Línia494	Verificar que s'incrementa una volta i es reinicien els sectors

CI:

En fer un merge a main s'executen els tests:

All checks have passed



1 successful check



Tests / tests (push) Successful in 40s

[Details](#)

(Si es miren els commits sortirà que això s'ha fet poques vegades, ja que es va començar a fer quan es va explicar a la sessió de pràctiques corresponent i per aquells temps, la meua pràctica estava pràcticament acabada. Igual que no acceptar el merge si no s'han validat els tests, ja que no vaig saber fins tard que si el repositori no és públic no funciona)

Es comprova la qualitat del codi d'acord amb unes normes preestablertes, per exemple el màxim tamany d'una línia:

Fitxer checkstyle.xml

```
<module name="Checker">
  <!-- Charset -->
  <property name="charset" value="UTF-8"/>

  <!-- Línia màxima -->
  <module name="LineLength">
    <property name="max" value="120"/>
    <property name="ignorePattern" value="^package.*|^import.*"/>
  </module>

  <!-- Comprovar final newline -->
  <module name="NewlineAtEndOfFile"/>

  <!-- Evitar tabs -->
  <module name="FileTabCharacter">
    <property name="eachLine" value="true"/>
  </module>

  <module name="TreeWalker">

    <!-- Noms -->
    <module name="TypeName"/>
    <module name="MethodName"/>
    <module name="ParameterName"/>
    <module name="LocalVariableName"/>
    <module name="ConstantName"/>

    <!-- Imports -->
    <module name="UnusedImports"/>
    <module name="RedundantImport"/>
    <module name="ImportOrder">
      <property name="option" value="top"/>
      <property name="groups" value="es,java,javax,org,com"/>
    </module>

  </module>
```

```
41      <!-- Espais i format -->
42      <module name="NoLineWrap"/>
43
44      <!-- Altres normes habituals -->
45      <module name="EmptyStatement"/>
46      <module name="EmptyBlock">
47      |   <property name="option" value="text"/>
48      </module>
49
50      <module name="Indentation">
51      |   <property name="basicOffset" value="4"/>
52      |   <property name="braceAdjustment" value="0"/>
53      |   <property name="caseIndent" value="4"/>
54      </module>
55
56      <module name="JavadocMethod"/>
57      <module name="JavadocType"/>
58  </module>
59 </module>
60
```

Un error en qualsevol acció provoca que no s'accepti el merge dins de main:

Branch protection rule

Branch name pattern *

main

Applies to 1 branch

main

Protect matching branches

☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☐ **Require approvals**
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

☐ **Dismiss stale pull request approvals when new commits are pushed**
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☐ **Require review from Code Owners**
Require an approved review in pull requests including files with a designated code owner.

☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☒ **Require status checks to pass before merging**
Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.

Fitxer CI.yaml:

```
1  name: Tests
2  on:
3    pull_request:
4      branches:
5        main
6    push:
7      branches:
8        main
9  jobs:
10   tests:
11     runs-on: ubuntu-latest
12     steps:
13       - name: Checkout code
14         uses: actions/checkout@v4
15       - name: Set up JDK 17
16         uses: actions/setup-java@v4
17         with:
18           distribution: 'temurin'
19           java-version: '17'
20       - name: Cache Maven Packages
21         uses: actions/cache@v4
22         with:
23           path: ~/.m2/repository
24           key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
25           restore-keys: |
26             ${{ runner.os }}-maven-
27       - name: Install dependencies
28         run: mvn install -DskipTests
29       - name: Code Quality (Checkstyle)
30         run: mvn checkstyle:check
31       - name: Run Tests
32         run: mvn test -Djava.awt.headless=true
```